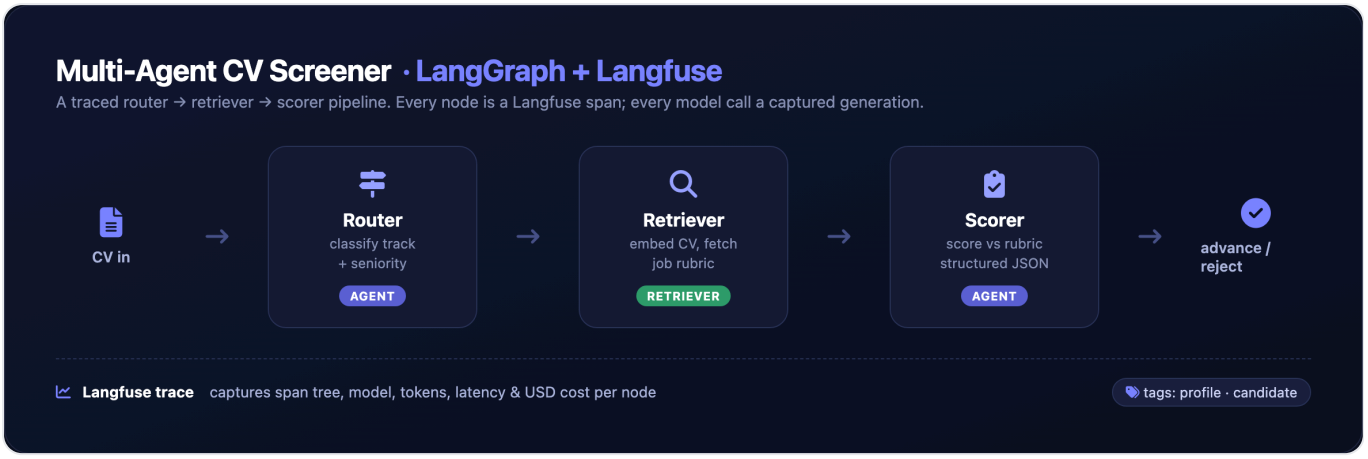
LLM AGENTS · OBSERVABILITY · OPTIMIZATION

Multi-Agent CV Screener — Trace-Driven Optimization

Adel Sherif · github.com/adelsherif8/cv-screener-langgraph · cv-screener-langgraph.vercel.app

A three-agent pipeline (router → retriever → scorer) built in LangGraph and fully instrumented with Langfuse. I read the production traces, identified the failure modes, fixed them with prompt and agent-config changes, and measured the before/after impact on a labeled evaluation set.

Architecture



Results — before / after

METRIC	BASELINE	OPTIMIZED	Δ
Decision accuracy	83.3%	**88.9%**	**+7%**
Scorer parse-failure	100%	**0%**	**-100%**
Latency / screen	5.83s	**3.67s**	**-37%**
Cost / 1,000 screens	$4.40	**$0.21**	**-95%**
Routing accuracy	100%	**100%**	**held**

Measured live through Langfuse over 18 labeled candidates × 2 profiles.

Failure modes found in the traces

A — Scorer output was 100% unparseable. The free-text scorer returned prose with no machine-readable score on every run; the graph defaulted the score and recovered the decision only by keyword-sniffing. Fixed with schema-constrained JSON output → parse failures 100% to 0%.

B — gpt-4o on every node drove cost and latency. Routing and rubric-scoring are bounded tasks; on gpt-4o the baseline cost ~$4.40 / 1,000 screens and ~5.8s/screen. Moving both nodes to gpt-4o-mini cut cost ~95% and latency ~37% with no accuracy loss.

Non-problem confirmed — routing was already correct. Routing held at 100% on both profiles, so the fix had to *protect* routing when downgrading the model, not repair it. Explicit prompts kept it at 100%.

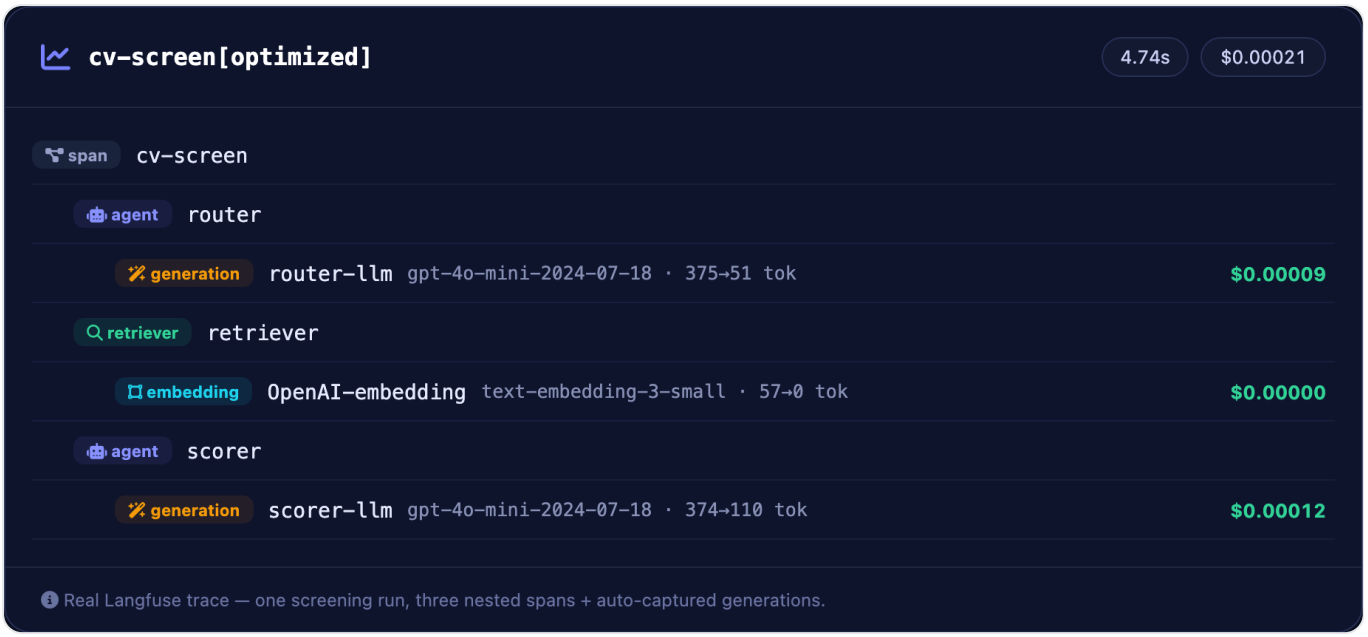
The fixes

FINDING	FIX
Unparseable scorer output	Schema-constrained JSON (decision / score / matched_must_haves / reasoning)
Cost & latency	gpt-4o → gpt-4o-mini on both LLM nodes
Routing under cheaper model	Explicit track definitions + disambiguation rule + temperature 0

All changes are prompt/agent-config only — no change to the task, data, or graph topology, so the gains are attributable to the fixes.

Trace evidence

Every screening run produces one Langfuse trace with a nested span tree and auto-captured generations (model, tokens, latency, USD cost per node):



Honest caveats

- Tokens rose ~41% (the optimized scorer injects the full rubric + schema), yet cost fell 95% — cost is tokens × model price, and gpt-4o-mini is far cheaper.
- The stricter optimized scorer newly over-rejects two senior-but-nontraditional candidates — the clearest target for the next iteration.

Reproduce

The full code, eval harness, and this case study are public: github.com/adelsherif8/cv-screener-langgraph — run python eval/run_eval.py to regenerate every number above.

Built by Adel Sherif · LangGraph · Langfuse · Python · OpenAI